

Implementacija i analiza genetskog algoritma za višestruko poravnanje sekvenci

Uvod	2
Multiple Sequence Alignment (MSA)	3
Implementacija grube sile i vremenska složenost	5
Genetski algoritmi	5
Implementacija GA za MSA	6
Reprezentacija jedinice	6
Operator selekcije	6
Operator ukrštanja	7
Operator mutacije	7
Elitizam	8
ClustalW algoritam	8
Poređenje GA i ClustalW	9
Rezultati	10
Zaključak	12
Literatura	13

Uvod

Multiple Sequence Alignment (MSA) predstavlja jedan od ključnih problema u oblasti bioinformatike, koji ima za cilj poravnanje više bioloških sekvenci na način koji maksimizira njihovu međusobnu sličnost. Ideja MSA-a je da se uoče homologne pozicije između različitih sekvenci, tj. mesta koja potiču od zajedničkog pretka, čime se mogu identifikovati konzervisani regioni, mutacije i evolutivne veze među organizmima.

Za razliku od pairwise alignmenta, koji upoređuje samo dve sekvence, višestruko poravnanje istovremeno uzima u obzir veći broj sekvenci, što omogućava mnogo bogatiju analizu bioloških odnosa. Međutim, upravo ta složenost čini problem MSA-a računski izuzetno zahtevnim. Ukupan broj mogućih poravnanja raste eksponencijalno sa brojem sekvenci i njihovom dužinom, pa se tačno rešenje može pronaći samo za mali broj kratkih sekvenci. Formalno, problem višestrukog poravnanja sekvenci spada u klasu NP-potpunih problema, što znači da ne postoji poznat efikasan algoritam koji uvek pronalazi optimalno rešenje u polinomnom vremenu. Zbog toga se u praksi koriste heuristički i aproksimativni pristupi, koji pokušavaju da pronađu dovoljno dobra, ali ne nužno optimalna rešenja.

Primene MSA-a obuhvataju širok spektar zadataka u bioinformatici i analizi podataka. Poravnanje više sekvenci omogućava prepoznavanje sličnosti i razlika između bioloških nizova, što se koristi za klasifikaciju, poređenje i analizu podataka o sekvencama. U praksi, MSA se primenjuje za prepoznavanje obrazaca, procenu srodnosti i pronalaženje zajedničkih karakteristika između različitih uzoraka. Takođe se koristi kao osnova za izgradnju modela koji predviđaju funkciju ili strukturu određenih molekula.

Zahvaljujući kombinovanju algoritama, statističkih metoda i podataka, MSA predstavlja važan alat koji povezuje analitičke pristupe iz računarstva sa potrebama istraživanja u biologiji i biotehnologiji.

Multiple Sequence Alignment (MSA)

MSA predstavlja proces poravnanja više bioloških sekvenci (npr. DNK, RNK ili proteina) sa ciljem da se identifikuju homologne pozicije — mesta koja su evolutivno ili funkcionalno srodna između sekvenci.

Ideja je da se sve sekvence predstave u tabelarnom obliku **jednake dužine**, tako što se po potrebi ubacuju **praznine** (*gap* karakter „-“) kako bi se slični ili identični simboli poravnali u istim kolonama. Na taj način se maksimizuje **ukupan skor poravnanja**, odnosno mera sličnosti između svih sekvenci.

Originalne sekvence:

1. PAWHE
2. HEAWY
3. AWHE

Poravnanje:

1. P-AWHE
2. HEAW-Y
3. -AW-HE

Sum-of-Pairs (SP) skor

Kvalitet MSA poravnanja najčešće se ocenjuje pomoću Sum-of-Pairs (SP) skora. Ovaj pristup sabira skorove svih mogućih parova sekvenci u poravnanju, po svakoj koloni, čime se dobija ukupna mera sličnosti između sekvenci.

Za svaku poziciju (k) u poravnanju i svaki par sekvenci (i, j) važe sledeća pravila:

- Ako su oba karaktera aminokiseline, njihova sličnost se ocenjuje prema vrednostima iz odgovarajuće supstitucione matrice.
- Ako je jedan karakter praznina („-“), primenjuje se kazna za prazninu (gap penalty).
- Ako su oba karaktera praznine, doprinos skoru je 0.

$$SP(A) = \sum_{i < j} \sum_k score(A[i][k], A[j][k])$$

gde je:

A – matrica poravnanja,

i, j – indeksi sekvenci u poravnanju,

k – indeks pozicije (kolone) u poravnanju.

Supstitucione matrice

Supstitucione matrice predstavljaju tabele koje određuju koliko je verovatno da se jedan simbol (npr. aminokiselina) zameni drugim u evolucijskom procesu. Vrednosti u matrici odražavaju učestalost i biološku prihvatljivost takvih zamena.

Pozitivne vrednosti označavaju česte i slične zamene (konzervativne supstitucije), dok negativne vrednosti ukazuju na retke ili nepovoljne zamene.

Najčešće korišćene supstitucione matrice uključuju:

- **BLOSUM62** – najčešće korišćena matrica za poravnanje proteinskih sekvenci; pogodna za sekvence umerenog stepena srodnosti (oko 60%).
- **BLOSUM45** – koristi se za poravnanja udaljenijih sekvenci (manje sličnosti).
- **BLOSUM80** – koristi se za blisko srodne sekvence (veća sličnost).

Izbor odgovarajuće matrice zavisi od vrste sekvenci koje se poravnavaju i očekivanog stepena njihove sličnosti.

Gap penalizacija

Tokom poravnanja, umetanje praznina se kažnjava pomoću tzv. gap penala, kako bi se sprečilo prekomerno umetanje „–“ karaktera.

U affine modelu razlikuju se dve vrste kazni:

- **Gap open penalty** – kazna za otvaranje nove praznine (obično između -10 i -15)
- **Gap extend penalty** – kazna za produžavanje već postojeće praznine (obično između -0.5 i -2)

Ukupna kazna za prazninu dužine L računa se kao:

$$\text{Gap}(L) = \text{gap_open} + (L - 1) \times \text{gap_extend}$$

Ovakav pristup omogućava da se otvaranje praznine strože kazni od njenog produžavanja, čime se postiže ravnoteža između dužine i broja praznina. Na taj način poravnanje postaje realističnije i stabilnije, jer se podstiče formiranje dužih, biološki verovatnijih praznina, umesto mnoštva kratkih prekida.

Implementacija fitness funkcije

Fitness funkcija implementirana pomoću pristupa **Sum-of-Pairs (SP)** skora, koji meri kvalitet višestrukog poravnanja sekvenci.

- **BLOSUM62 supstitucionu matricu**, kojom se ocenjuje sličnost između parova aminokiselina;
- **Affine gap penalizaciju**, kojom se uvodi kazna za praznine — *gap open penalty* (-10) i *gap extend penalty* (-1).

Implementacija grube sile i vremenska složenost

Brute-force pristup u projektu zasniva se na iscrpnom pretraživanju svih mogućih načina poravnanja sekvenci dodavanjem praznina na različite pozicije. Za svaku sekvencu generišu se sve varijante koje mogu nastati ubacivanjem do određenog broja praznina, čime se obezbeđuje potpuna pokrivenost svih mogućih kombinacija poravnanja. Nakon toga se sve sekvence međusobno kombinuju, tako da se svaka verzija jedne sekvence uporedi sa svim verzijama ostalih sekvenci. Za svako dobijeno poravnanje izračunava se ukupni skor (SP skor), a zatim se bira ono koje ima najveću vrednost, što predstavlja optimalno poravnanje.

Međutim i za veoma jednostavne primere desava se kombinatorna eksplozija i veliki broj potrebnih računanja. Za naredni primer i četiri dozvoljene praznine dobijamo skoro 400 hiljada različitih poravnanja koje je potrebno proveriti. Dok je za već za ovako mali primer potrebno oko 3 sekunde kako bi se izvršio.

Total number of alignments: 395136

Score: 10.0:

1. -PAWHE
2. HEAWY-
3. --AWHE

Genetski algoritmi

Genetski algoritmi predstavljaju optimizacione metode inspirisane principima prirodne selekcije i evolucije. Osnovna ideja ovih algoritama zasniva se na tome da se rešenja nekog problema posmatraju kao jedinke u populaciji, koje se tokom više generacija razvijaju i prilagođavaju, s ciljem postizanja što boljeg rezultata. Osnovna struktura genetskog algoritma obuhvata sledeće korake:

1. **Inicijalizacija** - generiše se početna populacija jedinki, najčešće nasumično ili na osnovu heurističkih pravila.
2. **Evaluacija** - za svaku jedinku izračunava se vrednost funkcije prilagođenosti (*fitnes*), koja meri kvalitet rešenja.
3. **Selekcija** - biraju se jedinke sa većom prilagođenošću koje će učestvovati u stvaranju nove generacije.
4. **Ukrštanje** - kombinuju se genetske informacije roditeljskih jedinki kako bi se formirali potomci.
5. **Mutacija** - uvode se nasumične promene u genom potomaka radi očuvanja raznovrsnosti populacije i izbegavanja lokalnih minimuma.
6. **Zamena** - formira se nova generacija, koja postaje osnova za sledeći ciklus evolucije.

Ovi koraci se ponavljaju sve dok nije ispunjen zadatak **uslov zaustavljanja** - najčešće dostizanje određenog broja generacija ili postizanje željenog nivoa kvaliteta rešenja.

Implementacija GA za MSA

Reprezentacija jedinke

Svaka jedinka u genetskom algoritmu predstavlja jedno moguće poravnanje sekvenci. Reprezentacija koju koristimo čuva pozicije praznina umesto celih poravnatih sekvenci, što je efikasnije po pitanju memorije. Struktura jedinke:

- `original_sequences`: Liste originalnih sekvenci (bez praznina)
- `max_gaps`: Maksimalan broj praznina po sekvenci
- `gap_positions`: Lista pozicija praznina za svaku sekvencu
- `code`: Finalno poravnanje (sekvence sa umetnutim prazninama)
- `fitness`: SP skor poravnanja

Primer kodiranja:

Originalna sekvenca: "PAWHE"

Gap pozicije: [0, 3]

Poravnata sekvenca: "-PAW-HE"

Kada se kreira nova jedinka, prvo se čuvaju originalne sekvence i maksimalan broj praznina. Zatim se za svaku sekvencu nasumično generiše broj praznina (između 0 i `max_gaps`) i njihove pozicije. Pozicije praznina se biraju nasumično iz mogućeg opsega, sortirane rastuće. Na kraju se iz originalnih sekvenci i pozicija praznina gradi kompletno poravnanje i izračunava fitness pomoću SP skora. Za svaku sekvencu i njene gap pozicije, algoritam iterira kroz ukupnu dužinu (originalna dužina + broj praznina). Ako je trenutna pozicija u skupu gap pozicija, dodaje se '-', inače se dodaje sledeći karakter iz originalne sekvence. Na kraju, sve sekvence se izjednačavaju na istu dužinu dodavanjem praznina na kraju kraćih sekvenci.

Operator selekcije

Turnirska selekcija

Turnirska selekcija nasumično bira `n` jedinki iz populacije. Od odabranih jedinki, vraća se ona sa najvišim fitness-om. Ovo omogućava kontrolisan selektivni pritisak - veći turnir daje prednost boljim jedinkama, dok manji turnir održava veći diverzitet.

- Jednostavan za implementaciju
- Kontrolisan selektivni pritisak (preko `tournament_size`)
- Ne zahteva sortiranje cele populacije

Rulet selekcija

Rulet selekcija prvo pomera sve fitness vrednosti u pozitivan opseg (ako postoje negativne vrednosti) oduzimanjem minimuma i dodavanjem 1. Zatim se sumiraju svi fitness i bira se nasumičan broj između 0 i te sume. Algoritam iterira kroz populaciju, kumulativno odajući fitness dok ne pređe izabrani broj - ta jedinka se vraća. Ovo daje proporcionalnu šansu selekcije baziranu na fitnessu.

- Stohastički pristup
- Bolje jedinke imaju veću šansu, ali nije garantovano
- Može imati problema sa preranom konvergencijom ako postoje prejake jedinke

Rank selekcija

Rank selekcija prvo sortira populaciju po fitness-u rastuće. Zatim dodeljuje rangove od 1 do N (gde je N veličina populacije). Koristi se rulet mehanizam nad rangovima umesto nad fitness vrednostima. To znači da razlika između najbolje i druge najbolje jedinke ima isti uticaj kao razlika između najlošije i druge najlošije jedinke, što sprečava dominaciju ekstremnih fitness vrednosti.

- Redukuje uticaj ekstremnih fitness vrednosti
- Održava konzistentni selektivni pritisak tokom generacija
- Sprečava preuranjenu konvergenciju

Operator ukrštanja

Ukrštanje kombinuje gap pozicije dva roditelja kako bi kreiralo dva potomka. Za svaku sekvencu u poravnanju, algoritam uzima gap pozicije iz oba roditelja. Ako oba roditelja imaju praznine za tu sekvencu, bira se nasumična tačka presek. Prvi potomak dobija praznine od početka do split point-a od prvog roditelja i ostatak od drugog roditelja (i obrnuto za drugog potomka). Ako jedan roditelj nema praznine za tu sekvencu, deca jednostavno razmenjuju gap pozicije. Nakon što se formiraju nove gap pozicije za sve sekvence, ponovo se grade kompletna poravnanja.

Operator mutacije

Add-Remove-Move mutacija

Za svaku sekvencu u poravnanju sa verovatnoćom p primenjuje se mutacija. Nasumično se bira tip mutacije: add, remove, ili move.

- **Add:** Ako broj praznina nije dostigao maksimum, bira se nasumična dostupna pozicija (koja već nije praznina) i dodaje se nova praznina
- **Remove:** Ako postoje praznine, nasumično se bira jedna praznina i uklanja
- **Move:** Nasumično se bira postojeća praznina koja se uklanja, zatim se bira nova dostupna pozicija gde se praznina dodaje

Add-Remove mutacija

Slično Add-Remove-Move mutaciji, ali bez opcije pomeranja praznina.

Elitizam

Elitizam je mehanizam koji garantuje da najbolje jedinke iz trenutne generacije neće biti izgubljene. Određeni broj najboljih jedinki (npr. 10 od 100) se direktno kopira u novu generaciju bez izmena, dok se preostale pozicije popunjavaju potomcima nastalim kroz selekciju, ukrštanje i mutaciju.

Prednosti:

- Garantuje očuvanje najboljeg rešenja
- Ubrzava konvergenciju

Mane:

- Može smanjiti diverzitet populacije
- Rizik preuranjene konvergencije

Preporučeno: 5-15% veličine populacije.

ClustalW algoritam

ClustalW je jedan od najpopularnijih progresivnih algoritama za višestruko poravnanje sekvenci (MSA). Algoritam gradi poravnanje postepeno, počevši od najbližih sekvenci i postupno dodajući udaljenije sekvence prema filogenetskom stablu.

1. Parno poravnanje: Izračunavaju se svi parni skorovi između sekvenci korišćenjem dinamičkog programiranja
2. Konstrukcija stabla: Na osnovu parnih skorova konstruiše se filogenetsko stablo pomoću UPGMA metode
3. Progresivno poravnanje: Sekvence se poravnavaju prema redosledu grananja u stablu, od lista prema korenu

Prednosti:

- Brz i efikasan za većinu praktičnih slučajeva
- Koristi biološki informisane supstitucione matrice (BLOSUM62)
- Adaptivne gap penalizacije

Mane:

- Greške napravljene u ranim fazama se propagiraju
- Nema garancije globalnog optimuma
- Može uvesti previše praznina kod divergentnih sekvenci

Poređenje GA i ClustalW

Za evaluaciju performansi koristili smo **BAlI**BASE (Benchmark Alignment dataBASE), jedan od najpoznatijih benchmark skupova za MSA. BAlIBASE sadrži ručno kurirana referentna poravnanja proteina organizovana u pet referentnih setova (RV11-RV50), gde svaki set testira različite karakteristike MSA algoritama:

- **RV11:** Ekvivalentne sekvence (sličan identitet)
- **RV12:** Sekvence sa različitim identitetom
- **RV20:** Familije sa “orphan” sekvencama
- **RV30:** Familije podeljene u podgrupe
- **RV40:** Sekvence sa N/C terminusnim ekstenzijama
- **RV50:** Interne insercije

Za eksperimente smo koristili: RV11 podskup sa maksimalnim brojem praznina od 50 po sekvenci

ClustalW je pokrenut sa parametrima prilagođenim implementaciji finte funkcije

- **Tip:** PROTEIN
- **Matrica:** BLOSUM62
- **Gap open penalty:** 10
- **Gap extend penalty:** 1

Genetski algoritam je pokrenut sa pet različitih konfiguracija

Konfiguracija	Selekcija	Mutacija	Pop.	Gen.	Tour. size	Mut. Prob.	Elitism
Tour AddRemMove	Turnir	Add Remove Move	100	50	10	0.10	10
Tour AddRem	Turnir	Add Remove	100	50	10	0.12	10
Roul AddRemMove	Rulet	Add Remove Move	100	50	-	0.10	10
Rank AddRemMove	Rank	Add Remove Move	100	50	-	0.10	10
Tour Large	Turnir	Add Remove Move	150	75	20	0.08	15

Rezultati

Konfiguracija	Prosečan skor	Prosečno vreme (s)
Tour_AddRemMove	-9184.93	14.368s
Tour_AddRem	-9008.47	14.376s
Roul_AddRemMove	-8989.32	14.347s
Rank_AddRemMove	-9147.68	14.347s
Tour_Large	-8745.72	32.208s
ClustalW	-4856.68	0.15s

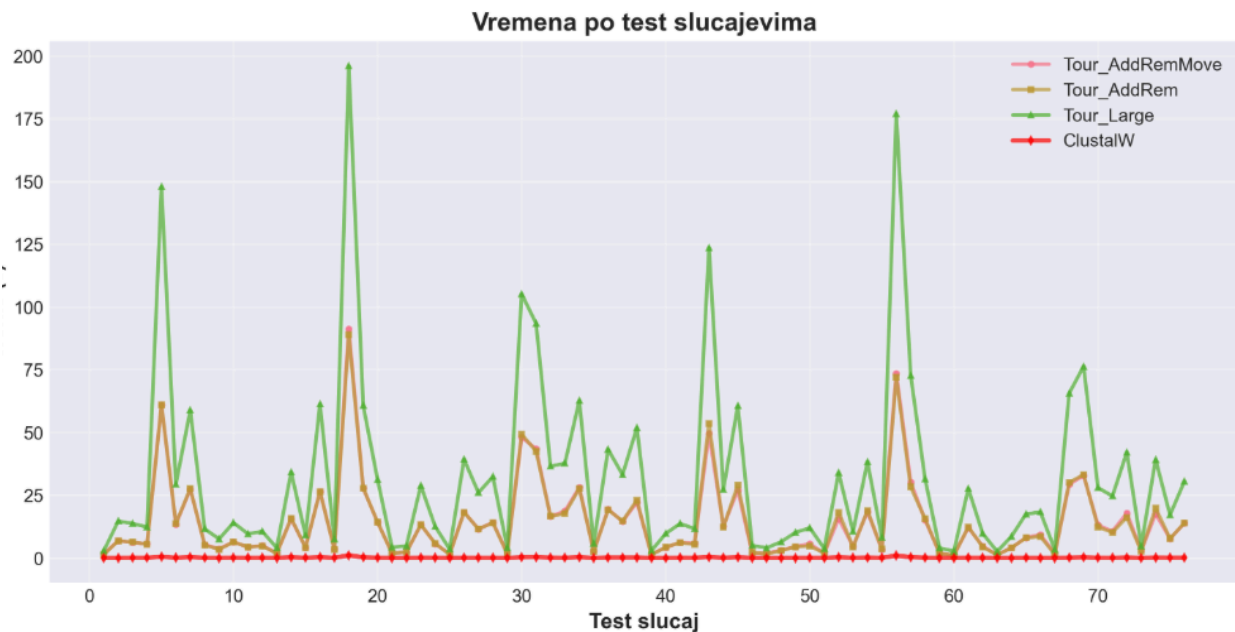
Tabela prikazuje prosečne vrednosti SP skora i vremena izvršavanja za različite konfiguracije genetskog algoritma (GA), zajedno sa referentnim ClustalW algoritmom. Najbolji rezultat postigao je ClustalW, sa prosečnim SP skorom od -4856.68 i prosečnim vremenom od svega 0.15 sekundi.

Sve GA varijante ostvaruju lošije skorove, ali zadržavaju približno jednako vreme izvršavanja od oko 14 sekundi, dok se Tour_Large izdvaja sa nešto boljim kvalitetom poravnanja (-8745.72) uz duže prosečno vreme od 32.2 sekunde.

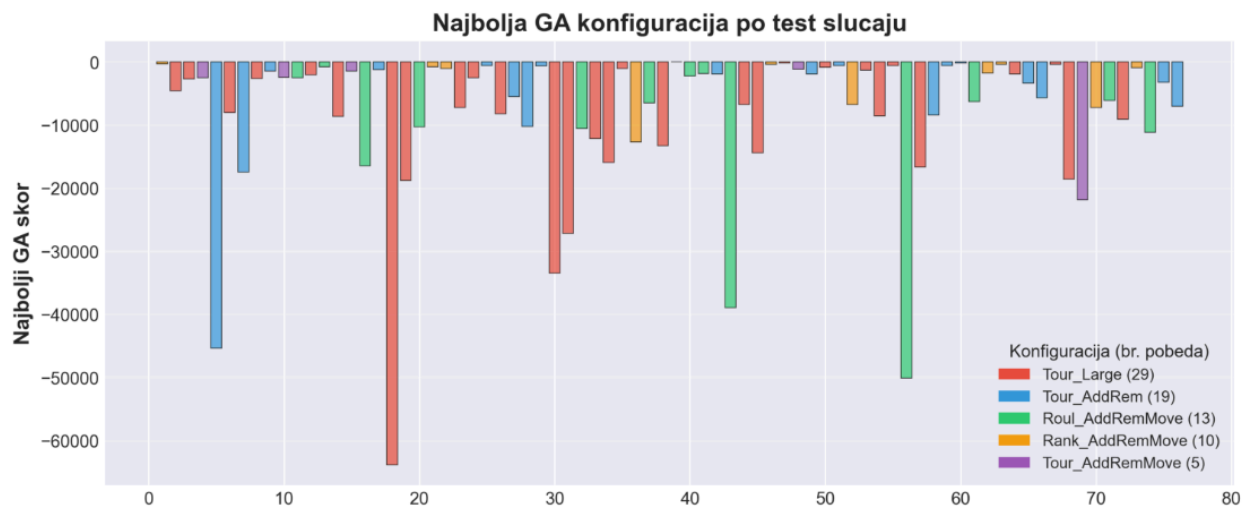
Male razlike među GA konfiguracijama dodatno potvrđuju njihovu stabilnost i konzistentnost.



Linije koje predstavljaju GA konfiguracije prate isti trend, ali su njihove vrednosti u proseku niže. Takav obrazac pokazuje da genetski algoritmi uspešno prepoznaju slične teškoće u poravnanju kao i ClustalW, ali sa slabijom preciznošću u finalnom rešenju.



Uporedni prikaz trajanja izvršavanja otkriva izrazitu razliku između pristupa. ClustalW se pokazuje kao izuzetno brz i stabilan - vreme izvršavanja je u svim slučajevima ispod jedne sekunde. Nasuprot tome, genetski algoritmi zahtevaju znatno više vremena: standardne konfiguracije se kreću između 10 i 30 sekundi, dok **Tour_Large** u pojedinim slučajevima dostiže i 180 sekundi. Ovaj trend potvrđuje da povećanje populacije i broja generacija poboljšava kvalitet rešenja, ali i značajno povećava troškove izvođenja.



Analiza raspodele najboljih rezultata po test slučajevima pokazuje da se **Tour_Large** najčešće pojavljuje kao najbolja konfiguracija (29 testova), zatim **Roul_AddRemMove** (13 testova), **Rank_AddRemMove** (10 testova), **Tour_AddRem** (9 testova) i **Tour_AddRemMove** (5 testova). Povećana inicijalna populacija i broj generacija dovele su do poboljšanja rezultata, ali na veliki uštrb efikasnosti.

Zaključak

U ovom radu smo implementirali i evaluirali genetski algoritam za problem višestrukog poravnanja sekvenci (MSA). Istražili smo različite operatore selekcije (turnirska, ruletska, rank) i mutacije (add-remove-move, add-remove), i uporedili performanse sa ClustalW algoritmom na BALiBASE test skupu.

Ključni nalazi:

1. GA pokazuje obećavajuće rezultate na manjim test slučajevima sa kraćim sekvencama, ali ima problema sa skaliranjem na veće instance problema
2. Turnirska selekcija sa Add-Remove-Move mutacijom daje najbolje rezultate među testiranim GA konfiguracijama, pokazujući dobar balans između eksploracije i eksploatacije
3. ClustalW je boji u većini test slučajeva zbog efikasnog progresivnog pristupa i pažljivo podešenih biološki-informisanih heuristika
4. Prostor pretrage MSA problema je ogroman (reda veličine 10^{14} za tipične sekvence), i naš GA istražuje samo zanemarljiv deo ovog prostora sa trenutnim parametrima
5. Računska složenost je glavni izazov - za postizanje konkurentnih rezultata sa ClustalW-om, GA bi zahtevao značajno veće populacije i broj generacija, što bi rezultiralo neprihvatljivim vremenima izvršavanja

Buduća istraživanja:

- Hibridni pristup: Kombinovanje GA sa lokalnom pretragom ili ClustalW inicijalizacijom
- Multi-objective optimizacija: NSGA-II ili MOEA/D za simultanu optimizaciju više ciljeva
- Adaptivni parametri: Dinamičko podešavanje `mutation_prob` i `tournament_size` tokom evolucije
- Paralelizacija: Implementacija island modela za brže izvršavanje

Literatura

- [1] **"AlineaGA: A Genetic Algorithm for Multiple Sequence Alignment."**
https://www.researchgate.net/publication/225980687_AlineaGA_A_Genetic_Algorithm_for_Multiple_Sequence_Alignment
- [2] **"ScienceDirect članak (PII: S0888754317300551)."** ScienceDirect.
<https://www.sciencedirect.com/science/article/pii/S0888754317300551>
- [3] **"The Phylogenetic Handbook."** <https://isu-molphyl.github.io/EEOB563/chapter3.pdf>
- [4] **"Clustal2 (ClustalW/ClustalX)"** <http://www.clustal.org/clustal2/>
- [5] **"BALiBASE - Benchmark Alignment Database."** <https://www.lbgi.fr/balibase/>